

Universidad de Costa Rica
CI-0117 Programación Paralela y Concurrente
Prof. Josef Ruczicka González
Tarea 1
Jimena Bejarano Sánchez C31074
Marina Castro Peralta C31886

1. Introducción

Este informe presenta el diseño, la implementación paralela con OpenMP y los resultados experimentales del simulador de la ecuación de calor 3D solicitado en la Tarea 1. Las mediciones se ejecutaron en el clúster Kabré, partición Kura.

2. Diseño e implementación

Estructura de datos

Se utilizan dos arreglos planos de tipo double, asignados dinámicamente con malloc, de tamaño N^3 cada uno (array_old y array_new). La conversión de coordenadas (i, j, k) a índice lineal se hace con la función:

$$\text{idx}(i, j, k, N) = i*N*N + j*N + k$$

Este ordenamiento (i exterior, k interior) favorece la localidad espacial en caché, ya que los vecinos en el eje k son contiguos en memoria. Al finalizar cada iteración, los punteros de array_old y array_new se intercambian, evitando copias explícitas y reduciendo el ancho de banda requerido por paso.

3. Paralelización con OpenMP

La paralelización se centra en la función jacobi_step, que constituye el núcleo de cómputo del programa. Se utiliza una única directiva OpenMP sobre los dos bucles externos:

```
#pragma omp parallel for collapse(2) schedule(static)

for (int i = 1; i < N-1; i++)
```

```

for (int j = 1; j < N-1; j++)

    for (int k = 1; k < N-1; k++)

        array_new[idx(i,j,k,N)] = (...) / 6.0;

```

La cláusula `collapse(2)` fusiona los bucles de `i` y `j` en un único espacio de iteración, lo que distribuye mejor la carga entre los hilos cuando `N` es moderado. El `schedule(static)` reparte bloques contiguos de iteraciones a cada hilo, lo cual es apropiado porque el trabajo por celda es uniforme.

Ausencia de condiciones de carrera

El esquema de Jacobi con dos arreglos garantiza la ausencia de condiciones de carrera: durante un paso, `array_old` solo se lee y `array_new` solo se escribe. Cada hilo trabaja sobre un subconjunto disjunto de índices de `array_new`, por lo que no existen escrituras simultáneas a la misma posición de memoria. El intercambio de punteros entre iteraciones ocurre fuera de la región paralela, en el bucle principal de `main`, y, por lo tanto, tampoco genera conflictos.

4. Metodología experimental

Las mediciones se ejecutaron en el clúster Kabré (CNCA), partición Kura, sobre un único nodo (`kura-1a.cnca`) reservando 16 cores mediante un job de SLURM. Cada configuración se repitió 5 veces para promediar el tiempo y reportar la desviación estándar. La medición del tiempo se realiza con `omp_get_wtime()` y cubre únicamente las `NUM_STEPS` iteraciones del bucle de Jacobi, excluyendo la inicialización y la escritura del archivo VTK, ya que en algunos casos de pruebas incluyendo la creación de archivos VTK ha durado hasta 230 segundos.

Se realizaron dos estudios:

- Strong scaling: tamaño del problema fijo ($N = 100$ y $N = 200$), variando el número de hilos en $\{1, 2, 4, 8, 16\}$.
- Weak scaling: trabajo por hilo aproximadamente constante; N escala como $p^{1/3}$ para mantener $N^3/p \approx 10^6$ puntos por hilo. Las parejas usadas fueron $(1, 100)$, $(2, 126)$, $(4, 159)$, $(8, 200)$, $(16, 252)$.

5. Resultados de strong scaling

La Tabla 1 resume los tiempos promedio, el speedup y la eficiencia para ambos tamaños de problema. El speedup se calcula como $T(1) / T(p)$, donde $T(1)$ es el tiempo de la versión paralela con un único hilo y $T(p)$ es el tiempo de la versión paralela con p hilos, ya que esta es la comparación más justa entre configuraciones de la misma versión.

N	Hilos	Tiempo (s)	Std (s)	Speedup	Eficiencia
100	1	1.221	0.010	1.00	100%
100	2	0.612	0.011	1.99	100%
100	4	0.312	0.006	3.91	98%
100	8	0.166	0.007	7.34	92%
100	16	0.160	0.002	7.63	48%
200	1	11.603	0.051	1.00	100%
200	2	6.229	0.008	1.86	93%
200	4	5.150	0.341	2.25	56%
200	8	3.607	0.013	3.22	40%
200	16	3.383	0.283	3.43	21%

Tabla 1. Resultados de strong scaling: tiempo promedio (5 repeticiones), speedup y eficiencia paralela.

Adicionalmente, se comparó el binario secuencial puro contra la versión paralela ejecutada con un único hilo, para verificar que la introducción de OpenMP no penaliza el caso serial:

- N = 100: serial = 1.222 s; paralelo (1 hilo) = 1.221 s. Diferencia ≈ -0.1 %.
- N = 200: serial = 11.883 s; paralelo (1 hilo) = 11.603 s. Diferencia ≈ -2.4 %.

El overhead de OpenMP con un hilo es despreciable (dentro del ruido de medición), lo que confirma que la versión paralela se comporta como la secuencial cuando `OMP_NUM_THREADS=1`.

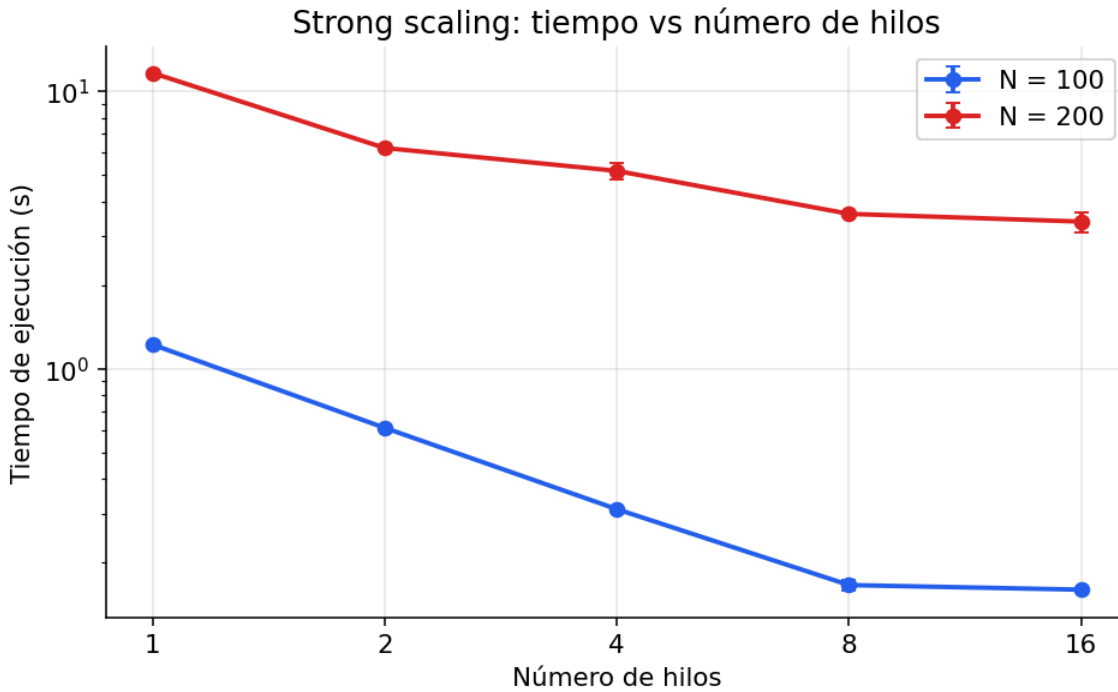


Figura 1. Tiempo de ejecución vs. número de hilos (escala log-log). Las barras de error corresponden a una desviación estándar.

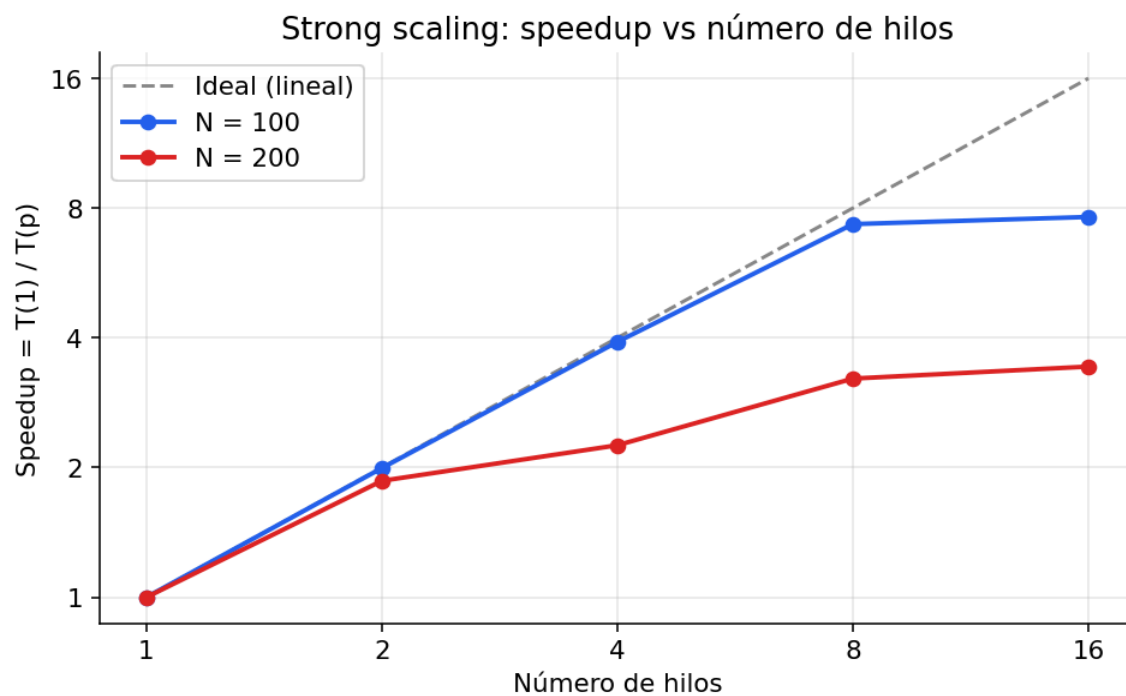


Figura 2. Speedup vs. número de hilos respecto al tiempo con un hilo. La línea discontinua indica el speedup ideal lineal.

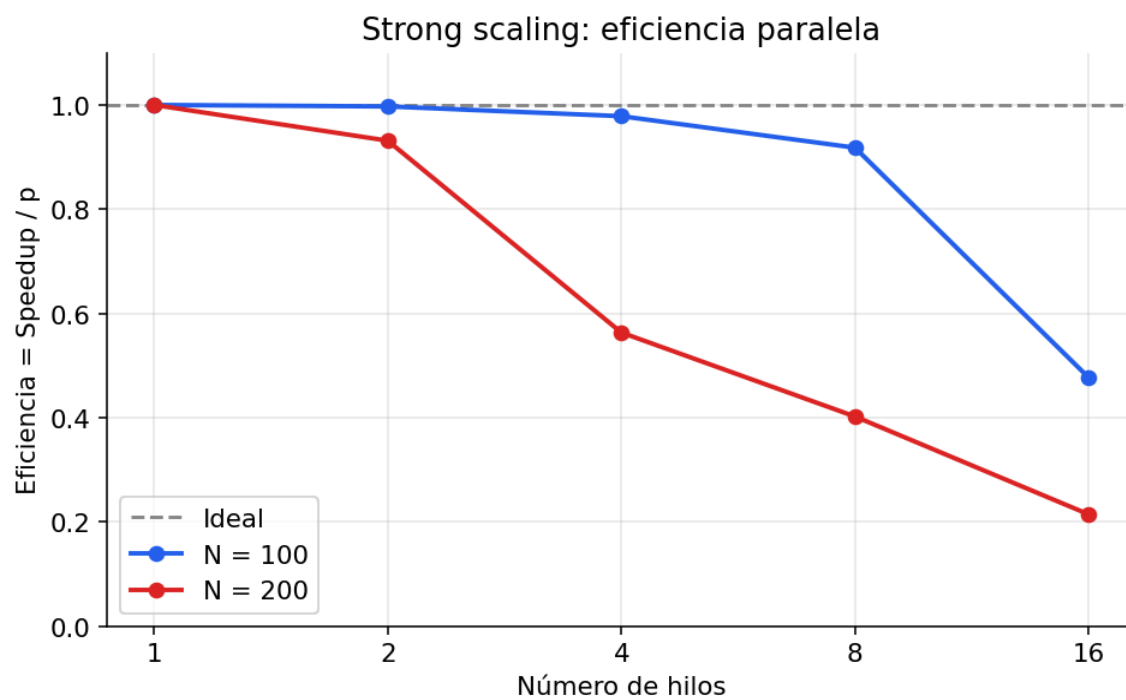


Figura 3. Eficiencia paralela vs. número de hilos. La línea discontinua marca la eficiencia ideal (1.0).

Discusión

Para $N = 100$, el scaling es prácticamente ideal hasta 8 hilos: el speedup obtenido es de $7.34\times$ con una eficiencia del 92 %. Al pasar a 16 hilos, se estanca en $7.63\times$ (eficiencia del 48 %), porque el problema es demasiado pequeño: cada hilo recibe $\sim 60\,000$ puntos, una cantidad insuficiente para amortizar el costo de la gestión del paralelismo.

Para $N = 200$, el speedup máximo es de solo $3.43\times$ con 16 hilos. La causa es que el estencil es memory-bound: 6 lecturas y 1 escritura de double por cada actualización, contra apenas 6 sumas y una división. El cuello de botella es el ancho de banda hacia memoria principal. La caché L3 de 30 MB se queda corta frente al tamaño combinado de los dos arreglos en $N = 200$ (≈ 128 MB), lo que obliga a accesos constantes a la memoria principal. Como el nodo (Intel Xeon Silver 4316, 20 núcleos físicos en un único socket) tiene núcleos físicos suficientes para todas las configuraciones probadas, descartamos contención por cambio de contexto u overhead de hyperthreading.

6. Resultados de weak scaling

En el estudio de weak scaling, el tamaño del problema crece con el número de hilos, de modo que cada hilo procesa aproximadamente la misma cantidad de puntos ($N^3 / p \approx 10^6$). En el caso ideal, el tiempo de ejecución debería permanecer constante. La Tabla 2 muestra los resultados.

Hilos	N	Tiempo (s)	Std (s)	Eficiencia weak
1	100	1.659	0.125	100%
2	126	1.932	0.009	86%
4	159	2.489	0.071	67%
8	200	3.578	0.155	46%
16	252	9.022	0.957	18%

Tabla 2. Resultados de weak scaling. Eficiencia weak = $T(1) / T(p)$.

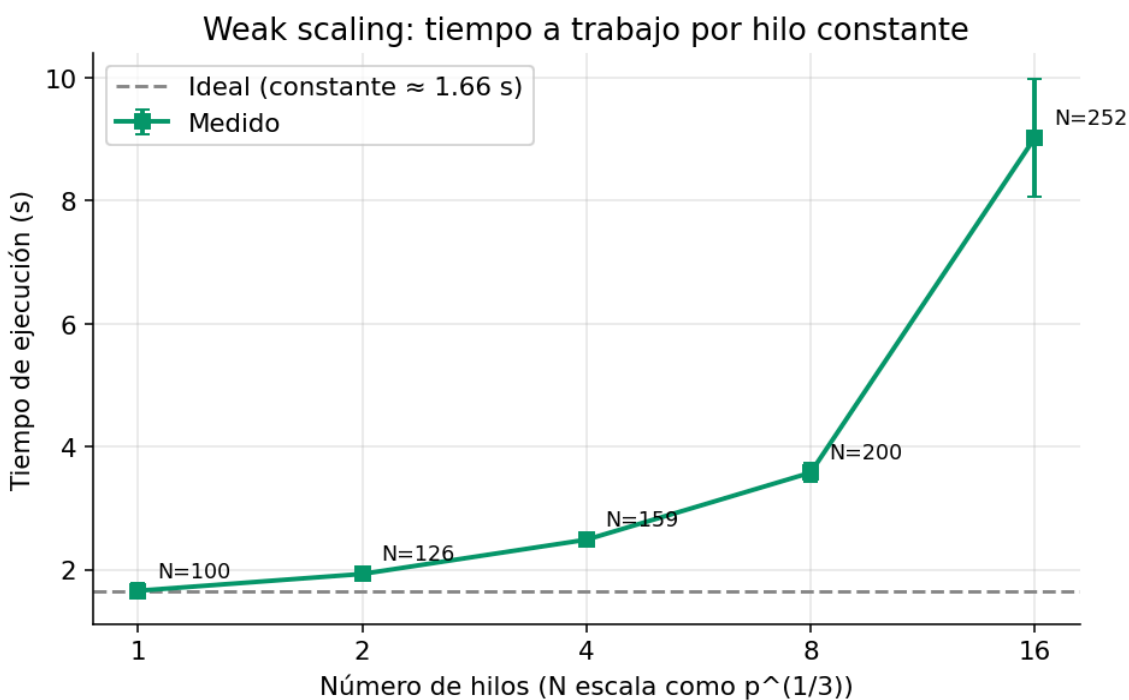


Figura 4. Tiempo de ejecución en régimen de weak scaling. La línea discontinua marca el tiempo ideal (constante).

Discusión

El tiempo crece de 1.66 s con un hilo a 9.02 s con 16 hilos, lo que da una eficiencia weak del 18 %. El salto más pronunciado se observa entre 8 y 16 hilos (de ~3.6 s a ~9.0 s, más del doble). Esto refuerza la conclusión de la sección anterior: el algoritmo es memory-bound y al aumentar simultáneamente los hilos activos y el tamaño total del problema, la presión sobre el ancho de banda crece más rápido que la capacidad agregada de cómputo.

Adicionalmente, los tamaños grandes ($N = 252$) generan mallas de aproximadamente 16 millones de puntos, lo que supera la capacidad de los niveles de caché y obliga a más accesos a la memoria principal. La variabilidad también es mayor en esta configuración (std de 0.96 s), lo que confirma la sensibilidad del rendimiento al estado del nodo y al subsistema de memoria.

7. Visualización en ParaView

El archivo `heat3d_output.vtk` generado por el programa contiene la distribución final de temperatura del cubo después de `NUM_STEPS` iteraciones. Se cargó en ParaView aplicando filtros de Slice y Volume Rendering para inspeccionar el interior del dominio.

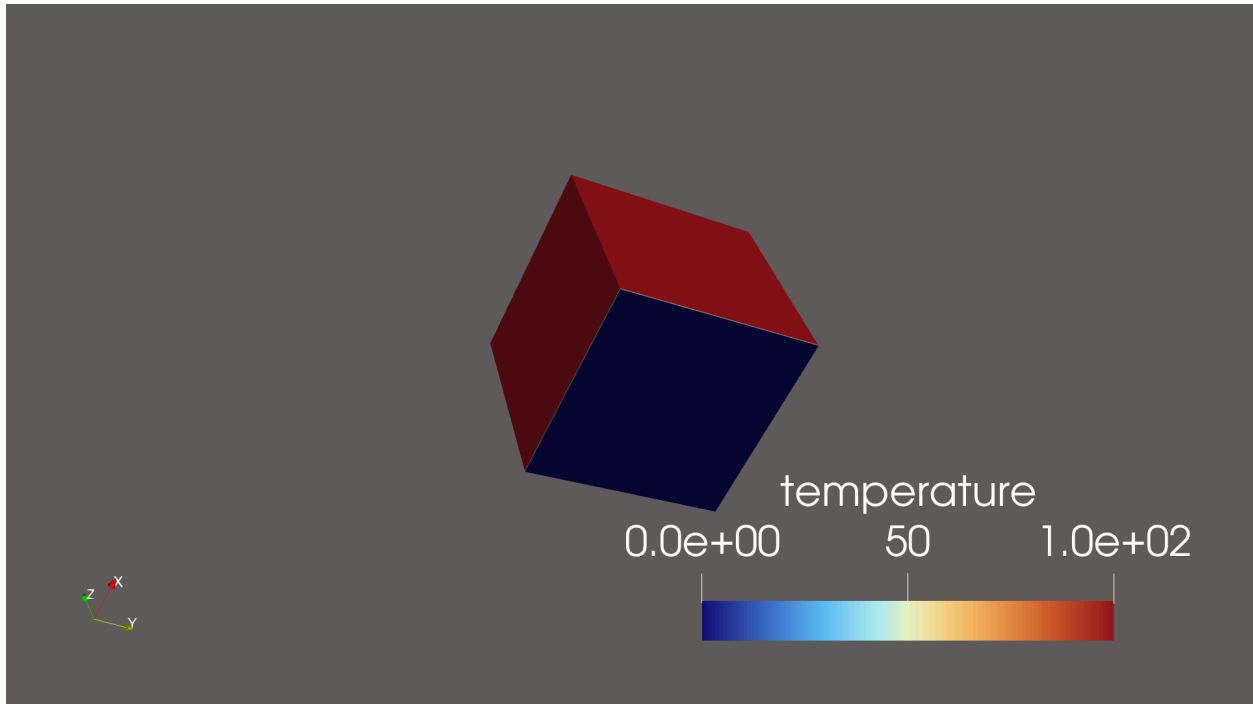


Figura 5. Vista general del cubo con el campo de temperatura. Las caras calientes (100 °C) aparecen en rojo y la cara fría (0 °C) en azul.

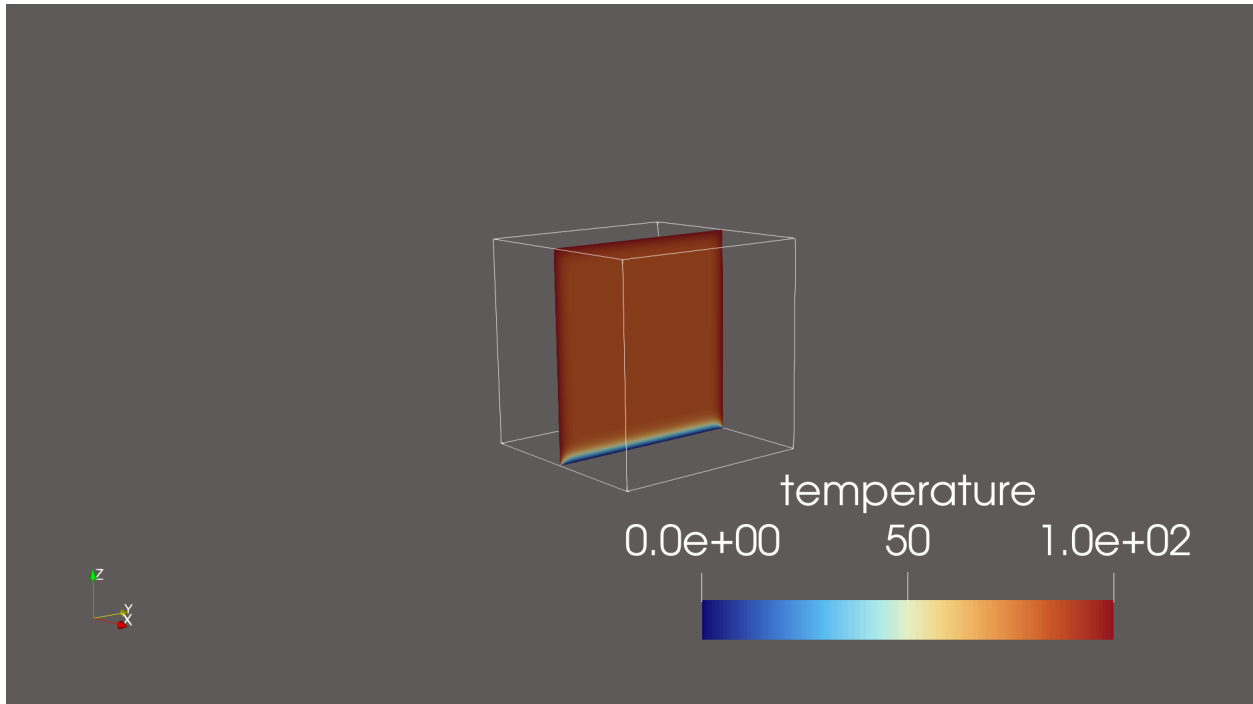


Figura 6. Corte por el plano medio del cubo, mostrando el gradiente interno de temperatura.

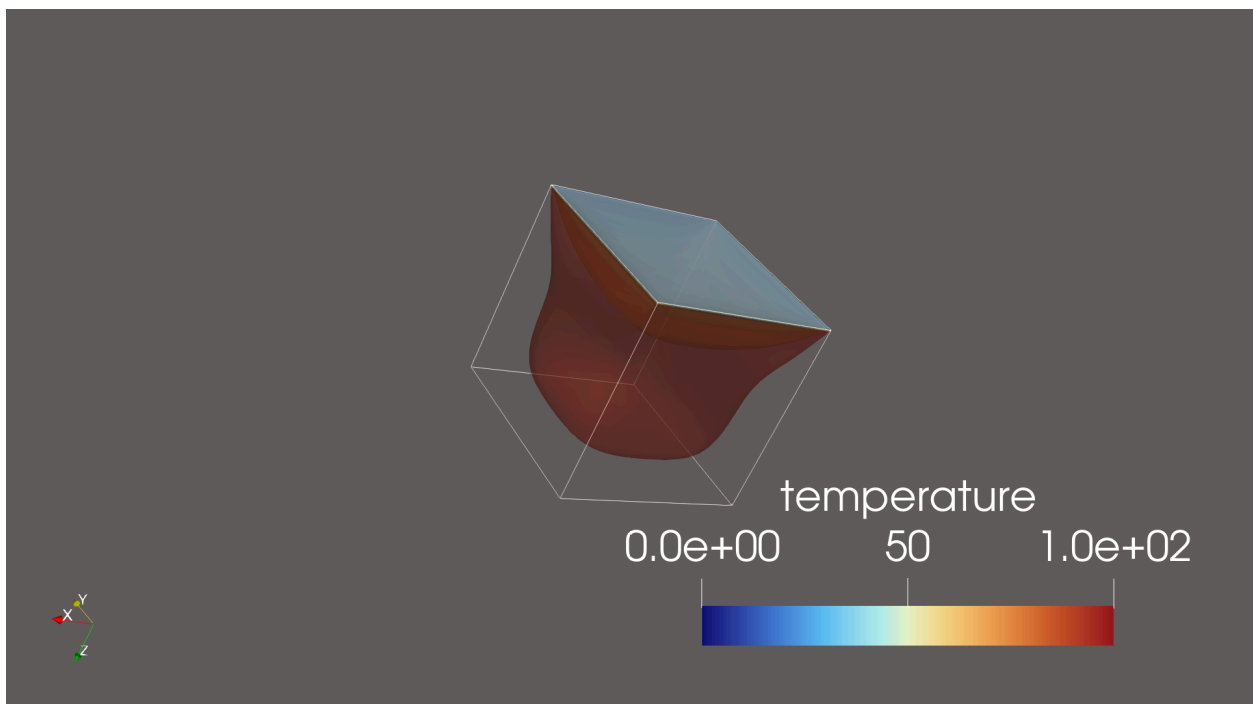


Figura 7. Renderizado volumétrico (o isosuperficies) que evidencia la distribución 3D del calor.

8. Reflexión sobre resultados, desafíos y posibles mejoras

Desafíos encontrados

- Variabilidad de las mediciones en el clúster: algunas configuraciones ($N = 200$ con 4 y 16 hilos) presentaron desviaciones estándar notoriamente altas, probablemente por contención con otros procesos en el mismo nodo. Tomar 5 repeticiones por configuración ayudó a estabilizar los promedios.
- Elegir el tamaño de problema adecuado: con $N = 100$ el problema resultó demasiado pequeño para 16 hilos. Esta sensibilidad al tamaño hace evidente la importancia de reportar resultados a varios valores de N .
- Diseñar el experimento de weak scaling correctamente: como el trabajo crece como N^3 , hubo que tomar N proporcional a $p^{(1/3)}$ para mantener trabajo/hilo constante, lo cual no es inmediato.

Posibles mejoras

- Aplicar cache blocking (tiling) al bucle del stencil para mejorar la reutilización de caché y aliviar la presión sobre la memoria principal.
- Experimentar con distintas combinaciones de schedule (dynamic, guided) y tamaños de chunk para evaluar el impacto.
- Incorporar un criterio de convergencia basado en la diferencia máxima entre iteraciones (umbral ϵ), para detener la simulación cuando se alcance el estado estacionario antes de NUM_STEPS, ahorrando trabajo.
- Verificar el programa con sanitizers (-fsanitize=address, -fsanitize=thread) para asegurar que no hay fugas de memoria ni condiciones de carrera, tal como se sugirió durante el curso.
- Generar archivos VTK intermedios cada cierto número de iteraciones para producir un video o GIF de la evolución del campo de temperatura.
- Extender el programa con condiciones de frontera variables (temperaturas distintas en cada cara) para estudiar configuraciones más realistas.